
QUESTION BANK

Data Structures (ETCCDS202)

B.Tech (CSE / AIML / DS) — Semester II

UNIT 1: Foundations & Algorithmic Analysis

Q1.

What is a Data Structure? Explain the need for data structures in software development with real-life examples.

Q2.

Differentiate between Abstract Data Type (ADT) and a concrete data structure. Give two examples of each.

Q3.

Explain Big-O, Big-Omega (Ω), and Theta (Θ) notations. How are they used to express algorithm efficiency?

Q4.

What are best-case, average-case, and worst-case complexities? Illustrate using linear search.

Q5.

Arrange the following complexities in increasing order of growth: $O(n^2)$, $O(\log n)$, $O(1)$, $O(n!)$, $O(n \log n)$. Draw a rough growth graph.

Q6.

Explain the concept of recursion. What is the role of a base case? Trace the recursive calls for computing factorial of 4.

Q7.

What is the Divide and Conquer paradigm? How does it differ from Dynamic Programming? Give one example of each.

Q8.

Analyze the time complexity of a nested loop: for i in range(n): for j in range(i , n): print(i , j) Justify your answer step by step.

Q9.

What is the Greedy algorithm approach? In what scenarios does Greedy fail to give an optimal solution? Provide an example.

Q10.

What is a call stack? How does it behave during recursive function execution? What happens if there is no base case?

UNIT 2: Linear Data Structures

Q1.

Compare arrays and linked lists based on memory allocation, access time, insertion, and deletion. When would you prefer one over the other?

Q2.

Explain Singly, Doubly, and Circular Linked Lists with diagrams. State one real-world application of each.

Q3.

Describe the insert and delete operations on a Singly Linked List. What is the time complexity of each operation?

Q4.

What is a Stack? Explain Push, Pop, and Peek operations with a real-world analogy. Describe how function calls use the stack.

Q5.

Explain how a Stack is used in expression evaluation. Convert the infix expression $A + B * C - D$ to postfix step by step.

Q6.

What is a Queue? Explain Enqueue and Dequeue. How does a Circular Queue differ from a Linear Queue and why is it more efficient?

Q7.

Trace BFS traversal on a graph with vertices $\{A, B, C, D, E\}$ and edges A-B, A-C, B-D, C-E. Show queue state at each step.

Q8.

What are overflow and underflow conditions in stacks and queues? How can they be handled in implementation?

Q9.

Explain a real-world scenario where a Queue would be a better choice than a Stack. Justify your answer with the operations involved.

Q10.

What is the difference between a static array and a dynamic array? Explain with memory diagrams how elements are stored in a 2D array.

UNIT 3: Sorting Algorithms

Q1.

What is sorting? Differentiate between stable/unstable and in-place/out-of-place sorting algorithms with examples.

Q2.

Explain Bubble Sort with a step-by-step trace on [64, 34, 25, 12, 22, 11]. State its best, average, and worst-case complexity.

Q3.

Trace Selection Sort on [29, 10, 14, 37, 13]. Show the array after each pass. Compare it with Insertion Sort for nearly-sorted data.

Q4.

Explain Merge Sort using the Divide and Conquer approach. Draw the recursion tree for [38, 27, 43, 3, 9, 82, 10].

Q5.

Explain the partitioning step in Quick Sort. Trace Quick Sort on [40, 20, 60, 10, 50, 30] using first element as pivot.

Q6.

Explain Heap Sort. What are the two phases involved? Why is Heap Sort preferred in memory-constrained environments?

Q7.

What is Counting Sort? Trace it on [4, 2, 2, 8, 3, 3, 1]. State its time and space complexity. When should it NOT be used?

Q8.

Explain Radix Sort. How does it handle multi-digit numbers? Trace it on [170, 45, 75, 90, 802, 24, 2, 66].

Q9.

Compare Quick Sort, Merge Sort, and Heap Sort in terms of: time complexity (all cases), space complexity, stability, and practical use.

Q10.

A developer needs to sort 1 million student records by roll number. Which sorting algorithm would you recommend and why? Consider time, space, and stability.

UNIT 4: Non-Linear & Advanced Data Structures

Q1.

Define a Binary Tree. Explain the following traversals with an example tree: In-order, Pre-order, Post-order. State one application of each.

Q2.

What is a Binary Search Tree (BST)? Insert [50, 30, 70, 20, 40, 60, 80] into an empty BST. Draw the resulting tree and explain the search operation.

Q3.

What is a Heap? Explain Min-Heap and Max-Heap properties. Show how to build a Max-Heap from [4, 10, 3, 5, 1] using heapify.

Q4.

What is the need for balanced trees like AVL and Red-Black Trees? Explain how AVL rotations (LL, RR, LR, RL) maintain balance.

Q5.

What is a Graph? Explain adjacency matrix and adjacency list representations with an example. Compare their space and time complexities.

Q6.

Explain DFS and BFS traversal algorithms with examples. For a graph {A-B, A-C, B-D, B-E, C-F}, trace both starting from A.

Q7.

What is Hashing? Design a simple hash function. Explain chaining and linear probing for collision resolution with examples.

Q8.

What is a Trie (Prefix Tree)? Show insertion of words ['cat', 'car', 'card', 'care', 'bat'] into a Trie. Explain how search works.

Q9.

What is a B-Tree? Why is it used in database indexing rather than a BST? Explain the properties of a B-Tree of order 3.

Q10.

Briefly explain the concept and application of: (i) Bloom Filters (ii) Segment Trees (iii) Skip Lists. When would you use each in industry?